CUDA-Accelerated Falling Sand Simulation with Real-Time OpenGL Rendering

A Project Component for

Parallel and Distributing Computing (UCS645)

By

Sr	Name	Roll No
1	Shreyash Tripathi	102303328
2	Harpuneet Singh	102483073
3	Aunish Kumar Yadav	102303306

Under the guidance of

Dr. Saif Nalband

(Assistant Professor, DCSE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(DEEMED TO BE UNIVERSITY)

PATIALA - 147004

MAY, 2025

Table of Contents

1 Introduction **1**

1.1 Background 1

1.2 Introduction to Problem Statement 1

2 Literature Review **3**

3 Research Gaps **5**

4 Problem Formulation **6**

5 Objectives **6**

6 Methodology **8**

6.1 Ping-Pong Buffer Architecture 9

6.2 Atomic Conflict Resolution 9

6.3 Gravity Simulation – SimulateParticlesKernel 10

6.4 Eruption / Blast Effect 10

6.5 Colour Encoding 11

6.6 Thread Hierarchy 11

7 Datasets **12**

8 Results and Discussion **13**

8.1 Correctness of Gravity Simulation 13

8.2 Race Condition Elimination 13

8.3 Eruption Effect 13

8.4 Colour Cycling 13

8.5 Performance 14

8.5.1 Block Size 8×8 (64 threads/block) 14

8.5.2 Block Size 16×16 (256 threads/block) 15

8.5.3 Block Size 32×8 (256 threads/block) 16

8.5.4 Cross-Block-Size Comparison 16

8.6 Limitations and Future Work 17

References **19**

List of Figures

1 Performance profile for block size 8×8. 15

2 Performance profile for block size 16×16. 15

3 Performance profile for block size 32×8. 16

4 Direct comparison of FPS and frame time across all three block size configurations. 17

List of Tables

1 CPU vs GPU characteristics for simulation workloads 2
2 Simulation Configuration 2
3 Summary of Literature Review 4
4 Research gaps and corresponding contributions of this project 5
5 Notation used in the problem formulation 7
6 Project objectives and their corresponding implementation mechanisms . . 8
7 Thread Block Configurations 8
8 Per-frame kernel pipeline: phases, kernels, inputs, and outputs 9
9 Runtime Memory Structures 12
10 Occupancy characteristics for each block size configuration 14
11 Qualitative comparison of block size configurations 17
12 Known limitations and proposed future improvements 18

1 Introduction

1.1 Background

Particle simulation is a foundational problem in computer graphics, scientific computing, and game development. Simulating the behaviour of granular materials such as sand, water, or powder requires evaluating thousands to millions of individual particle interactions every frame. On traditional single-threaded CPUs, this quickly becomes a bottleneck. The emergence of General-Purpose GPU Computing (GPGPU), and specifically NVIDIA's CUDA (Compute Unified Device Architecture) platform, has made it possible to offload such massively parallel workloads to the GPU, where thousands of threads can execute simultaneously [1].

Cellular automata (CA) are a class of computational model in which a grid of cells evolves according to a fixed set of local rules. The falling sand game is a well-known application of 2D cellular automata, where each cell represents either an empty space or a particle type (e.g., sand, water, fire), and particles move based on simple gravity-driven rules applied at every time step [2]. When this paradigm is combined with the parallel throughput of CUDA and the real-time display capabilities of OpenGL, the result is a responsive, visually rich simulation capable of handling an 800x600 grid (480,000 cells) at interactive frame rates.

A CPU typically executes tasks sequentially with limited cores (4-16), whereas a GPU can execute thousands of lightweight threads simultaneously. This makes GPUs particularly suitable for grid-based simulations such as cellular automata.

At a resolution of 800x600, the simulation involves 480,000 cells per frame. At 60 FPS, this corresponds to approximately 28.8 million cell updates per second.

1.2 Introduction to Problem Statement

Real-time particle simulations that are both physically plausible and visually interactive present a significant computational challenge. A naive CPU implementation must iterate over every cell in the grid sequentially each frame, which scales poorly as resolution grows. The project addresses this challenge by mapping the simulation to the GPU using CUDA, where each thread is responsible for exactly one cell, allowing all 480,000 cells to

Property	**CPU vs GPU**
Cores	4–16 (CPU) vs 1000s (GPU)
Execution Model	Sequential (CPU) vs Massively Parallel (GPU)
Suitable For	General Logic (CPU) vs Grid/Data-Parallel Tasks (GPU)
Latency	Low (CPU) vs Higher (GPU)
Throughput	Moderate (CPU) vs Very High (GPU)

Table 1: CPU vs GPU characteristics for simulation workloads

be evaluated in parallel [3].

A critical sub-problem in parallel cellular automata simulation is *race conditions*: when two neighbouring particles simultaneously decide to move into the same empty cell, data corruption occurs. This project solves that problem using CUDA atomic operations, specifically `atomicCAS` (Compare-And-Swap), which guarantees that only one particle can claim a target cell per time step, even when tens of thousands of threads are competing simultaneously.

The simulation also integrates real-time mouse input, enabling the user to spawn sand by holding the left mouse button and trigger a physics-based eruption/blast effect by holding the right mouse button, all while maintaining smooth interactive performance.

Parameter	Value
Grid Size	800 Ã 600
Total Cells	480,000
Particle Type	Sand
Simulation Model	Cellular Automata
GPU Used	(Your GPU name)
Programming Model	CUDA + OpenGL

Table 2: Simulation Configuration

2 Literature Review

References	**Concepts Highlighted**	**Technique(s) Used**	**Significance / Proposal**	**Limitations**
[3]	GPU architecture; parallel thread execution; CUDA programming model	CUDA C/C++ kernels; thread-/block/grid hierarchy	Provides the theoretical and practical foundation for writing massively parallel GPU code; explains the performance gap between CPU and GPU	Requires dedicated NVIDIA hardware; portability to AMD GPUs not addressed
[4]	CUDA memory model; atomic operations; device synchronisation	`cudaMalloc`, `cudaMemcpy`, `atomicCAS`	Hands-on introduction to GPU memory management and thread-safe programming patterns directly applicable to particle simulation	Examples focus on scientific computing; game-simulation use cases are limited

[2]	Granular material simulation; cellular automata	2D CA rules for powder and sand dynamics	Establishes that CA is a suitable model for granular physics and validates falling-sand rules against real powder behaviour	CPU-only; no discussion of parallelisation or GPU acceleration
[1]	Scalable parallel programming; warp execution; occupancy	SIMT (Single Instruction Multiple Threads) model	Explains CUDA's execution model, helping developers understand why atomic operations are necessary for conflict-free parallel writes	Highly theoretical; implementation details left to the developer
[5]	Real-time rendering pipeline; texture mapping; double buffering	OpenGL 2D texture upload; quad rendering; GLFW windowing	Describes the OpenGL pipeline used to display the CUDA colour buffer as a full-screen texture each frame	Modern OpenGL (4.x) approaches such as PBOs would be more efficient for CUDA–OpenGL interoperability

Table 3: Summary of Literature Review

3 Research Gaps

The following gaps were identified from the literature review:

- ***Absence of Atomic-Safe Parallel CA Implementations***: Most published cellular automata simulations for granular materials are CPU-based and sequential. Literature on GPU-accelerated CA does not adequately address the race-condition problem that arises when thousands of threads simultaneously attempt to move particles into shared neighbouring cells.

- ***Lack of CUDA–OpenGL Interoperability Discussion***: Existing works treat GPU compute and GPU rendering as separate concerns. The efficient pathway of writing simulation output directly into an OpenGL pixel buffer object (PBO) from a CUDA kernel, thereby avoiding a costly device-to-host memory transfer every frame, is not covered in the surveyed literature.

- ***No Interactive Force / Eruption Models in Parallel Simulations***: Published falling-sand implementations focus only on passive gravity. The application of mouse-driven repulsion forces and ray-cast eruption mechanics in a GPU-parallel setting has not been explored in the literature, representing an opportunity to combine interactive physics with massive parallelism.

Gap	**Issue in Literature**	**This Project's Contribution**
Atomic-Safe Parallel CA	No GPU CA handles concurrent write conflicts	Uses `atomicCAS` claims buffer for race-free moves
CUDA–OpenGL Interop	Compute and rendering treated separately	Shared colour buffer written by CUDA, read by OpenGL
Interactive Force Models	Only passive gravity modelled	Mouse-driven ray-cast blast effect implemented

Table 4: Research gaps and corresponding contributions of this project

4 Problem Formulation

The core problem addressed in this project is the real-time parallel simulation of granular particle dynamics (falling sand) on a 2D grid of resolution $W \times H$ ($800 \times 600 = 480{,}000$ cells), subject to the following constraints:

- Each cell $c_{x,y}$ at time step t holds either an `EMPTY` value (0) or a packed 32-bit RGBA colour representing a sand particle.
- At each time step, every non-empty cell must evaluate a set of gravity rules and attempt to move to a lower-energy position (downward or diagonally downward).
- No two particles may occupy the same cell after a time step – i.e., every successful move must be mutually exclusive.
- The simulation must produce a rendered frame at interactive rates ($\geq$30 FPS) on a consumer GPU.
- User input (mouse position and button state) must be incorporated each frame to spawn new particles and apply force-based displacement.

Formally, let $G^t \in \mathbb{Z}^{W \times H}$ denote the grid state at time t. The simulation computes G^{t+1} from G^t by applying a parallel transition function $\mathcal{F}$ to all cells simultaneously:

$$G^{t+1} = \mathcal{F}(G^t, \mathbf{m}_t, b_t)$$

where $\mathbf{m}_t = (m_x, m_y)$ is the mouse cursor position at time t and $b_t \in \{0, 1, 2\}$ encodes the mouse button state (none, left, right). The challenge lies in implementing $\mathcal{F}$ in a race-condition-free, fully parallel manner on the GPU.

5 Objectives

- To implement a GPU-parallel falling-sand cellular automaton using CUDA that simulates gravity-driven particle movement across a 480,000-cell grid at interactive frame rates.

Symbol	Description	Value / Domain
$W \times H$	Grid dimensions	800×600
G^t	Grid state at time step t	$\mathbb{Z}^{W \times H}$
$\mathbf{m}_t$	Mouse cursor position	$(m_x, m_y) \in [0, W] \times [0, H]$
b_t	Mouse button state	$\{0, 1, 2\}$
$\mathcal{F}$	Parallel transition function	GPU kernel

Table 5: Notation used in the problem formulation

- To eliminate race conditions in parallel cell updates using CUDA atomic operations (`atomicCAS`), ensuring correctness of the simulation under fully concurrent thread execution.
- To integrate real-time mouse interaction for particle spawning (left-click) and physics-based eruption/blast displacement (right-click) using ray-cast repulsion vectors.
- To render the simulation output in real-time using OpenGL and GLFW, employing a ping-pong double-buffer strategy to decouple simulation and rendering.
- To produce visually rich output by encoding particle colour as a time-varying packed RGB value using phase-shifted sine wave colour cycling.

Sr. No.	Objective	Key Mechanism
1	GPU-parallel CA simulation at interactive rates	CUDA kernel, 480,000 threads
2	Race-condition-free parallel updates	`atomicCAS` claims buffer
3	Real-time mouse interaction	Left-click spawn; right-click blast
4	Real-time rendering via OpenGL	Ping-pong double buffer + GLFW
5	Visually rich colour output	Phase-shifted sine wave RGB cycling

Table 6: Project objectives and their corresponding implementation mechanisms

6 Methodology

Configuration	Threads per Block	Total Threads
8x8	64	480,000
16x16	256	480,000
32x8	256	480,000

Table 7: Thread Block Configurations

The system is structured as a pipeline of four CUDA kernels executed every frame, connected by a ping-pong buffer architecture. The overall data flow is as follows:

1. **Input Phase** – `AddSandKernel`: Spawns sand particles at the mouse position.
2. **Simulation Phase** – `SimulateParticlesKernel`: Applies gravity and eruption physics.
3. **Render Phase** – `RenderToColorKernel`: Converts the grid to an RGBA pixel buffer.
4. **Display Phase** – OpenGL texture upload and full-screen quad render.

Phase	Kernel	Input	Output
1	`AddSandKernel`	Mouse position & button	Updated `d_gridInput`
2	`SimulateParticlesKernel`	`d_gridInput`, `d_claims`	`d_gridOutput`
3	`RenderToColorKernel`	`d_gridOutput`	`d_colorBuffer`
4	OpenGL display	`h_colorBuffer`	Screen frame

Table 8: Per-frame kernel pipeline: phases, kernels, inputs, and outputs

6.1 Ping-Pong Buffer Architecture

To avoid reading and writing the same memory simultaneously, two grids are maintained: `d_gridInput` and `d_gridOutput`. Each frame, the simulation reads from `d_gridInput` and writes results to `d_gridOutput`. After the frame, the pointers are swapped at zero cost (no data copy). This is a standard GPU double-buffering technique [4].

6.2 Atomic Conflict Resolution

A key challenge in parallel CA simulation is that multiple threads may select the same target cell. A dedicated `d_claims` buffer (one `unsigned int` per cell, cleared to zero each frame) acts as a lock array. The device function `AtomicCheckUnclaimed` uses `atomicCAS` to atomically transition a claim slot from 0 to 1. Only the first thread to succeed proceeds with the move; all others fall back.

The atomic claim mechanism is expressed as:

$$\text{previous} = \text{atomicCAS}(\&\text{claims}[i],\ 0,\ 1)$$

$$\text{claim_granted} = (\text{previous} == 0)$$

Algorithm 1 AtomicCheckUnclaimed – GPU Device Function

Require: claims array C, target cell index i

Ensure: Returns **true** if this thread is the first to claim cell i

1: prev $\leftarrow$ atomicCAS($C[i]$, 0, 1)

2: **return** (prev = 0)

6.3 Gravity Simulation – SimulateParticlesKernel

Each CUDA thread is assigned to one (x, y) cell. If the cell is empty the thread exits immediately. Otherwise the particle attempts to move in the following priority order:

Algorithm 2 Particle Gravity Transition

Require: Grid G^t, claims C, current position (x, y), oddFrame flag

Ensure: Updates G^{t+1} for cell (x, y)

```
if G^t[x, y] = EMPTY then return
end if
down ← (x, y − 1)
diagA ← (x + oddFrame · 1 + (1 − oddFrame) · (−1),  y − 1)
diagB ← (x − oddFrame · 1 + (1 − oddFrame) · 1,  y − 1)
for target ∈ {down, diagA, diagB} do
    if G^t[target] = EMPTY and AtomicCheckUnclaimed(C, target) then
        G^{t+1}[x, y] ← EMPTY
        G^{t+1}[target] ← G^t[x, y]
        return
    end if
end for
```

The `oddFrame` flag alternates each step, biasing diagonal preference left/right on alternate frames to prevent asymmetric pile formation.

6.4 Eruption / Blast Effect

When the right mouse button is held, each sand particle within the brush radius of the mouse cursor is subject to an outward blast. A unit direction vector $\hat{n}$ is computed from the mouse position $\mathbf{m}$ to the particle position $\mathbf{p}$:

$$\hat{n} = \frac{\mathbf{p} - \mathbf{m}}{\|\mathbf{p} - \mathbf{m}\|} + (0,\ 0.2)^T$$

The additive $(0, 0.2)$ term provides an upward bias so that blasted particles travel in an arc rather than purely horizontally. A ray-cast is then performed along $\hat{n}$, stepping

pixel by pixel up to 200 cells, and the first empty cell encountered is claimed atomically as the particle's new position.

6.5 Colour Encoding

Each particle stores its colour packed into a 32-bit integer as `0xAARRGGBB`. Colour is assigned at spawn time using three phase-shifted sine waves driven by the simulation step counter t, producing a smoothly cycling rainbow effect:

$$r(t) = \left\lfloor 255 \cdot \frac{\sin(\omega t) + 1}{2} \right\rfloor, \quad g(t) = \left\lfloor 255 \cdot \frac{\sin(\omega t + \frac{2\pi}{3}) + 1}{2} \right\rfloor, \quad b(t) = \left\lfloor 255 \cdot \frac{\sin(\omega t + \frac{4\pi}{3}) + 1}{2} \right\rfloor$$

where $\omega = 0.05$ rad/step. Sand poured at different moments therefore has visually distinct colours, making the temporal history of the simulation visible in the pile.

6.6 Thread Hierarchy

Threads are arranged in 2D blocks of $8 \times 8 = 64$ threads. The number of blocks is computed to cover the full grid:

$$\text{numBlocks}_x = \left\lceil \frac{W}{8} \right\rceil, \qquad \text{numBlocks}_y = \left\lceil \frac{H}{8} \right\rceil$$

For an 800×600 grid this gives $100 \times 75 = 7500$ blocks and a total of $480,000$ threads, one per cell.

7 Datasets

This project is a real-time simulation and does not rely on external datasets. The "data" generated and consumed by the system is the simulation grid itself, which is fully procedurally generated at runtime through user interaction and the physics rules. The table below summarises the memory structures used in place of traditional datasets.

Buffer Name	**Description**	**Size (800x600)**	**Type**	**Location**
`d_gridInput`	Particle state grid – read each frame; each cell stores either 0 (EMPTY) or a packed RGBA integer	1.92 MB	`int [480000]`	GPU VRAM
`d_gridOutput`	Particle state grid – written each frame; swapped with input after simulation	1.92 MB	`int [480000]`	GPU VRAM
`d_claims`	Atomic lock array; one `unsigned int` per cell; cleared to 0 before each simulation step	1.92 MB	`uint [480000]`	GPU VRAM
`d_colorBuffer`	RGBA pixel buffer written by `RenderToColorKernel`; copied to CPU for OpenGL texture upload	1.92 MB	`uchar4 [480000]`	GPU VRAM
`h_colorBuffer`	CPU-side mirror of the colour buffer; uploaded to OpenGL texture each frame	1.92 MB	`uchar4 [480000]`	RAM

Table 9: Runtime Memory Structures

8 Results and Discussion

8.1 Correctness of Gravity Simulation

The simulation correctly implements the three-priority gravity rule: particles fall straight down first, then diagonally. Sand piles form stable angles of repose and do not collapse spuriously. The `oddFrame` alternation eliminates the directional bias that would otherwise cause all piles to lean consistently to one side.

8.2 Race Condition Elimination

Without the `atomicCAS` claim mechanism, two particles competing for the same cell would produce visual artefacts such as particles appearing and disappearing randomly. With the claims buffer in place, all movements are mutually exclusive and the grid remains consistent across all 480,000 concurrent threads. The atomic operation adds a small but acceptable overhead compared to the alternative of serialising updates.

8.3 Eruption Effect

The right-click blast effect successfully displaces buried sand to the surface by ray-casting up to 200 pixels in the outward direction with an upward bias of 0.2 units. Sand particles erupt visibly from within a pile and land at the surface, creating a visually convincing explosion without requiring any additional physics engine – purely emergent from the existing atomic-move mechanism applied along a computed ray direction.

8.4 Colour Cycling

The sine-wave colour packing produces smoothly transitioning rainbow colours that change with simulation time. This makes the temporal structure of sand deposits immediately visible: sand poured at time t_1 has a different hue from sand poured at t_2, acting as a natural 'timestamp' within the pile.

8.5 Performance

Running on a modern consumer GPU, the simulation maintains interactive frame rates ($\geq$30 FPS) at 800×600 resolution. The dominant cost is the `cudaMemcpy` of the colour buffer from device to host each frame (1.92 MB), which could be eliminated in a future version using CUDA–OpenGL Pixel Buffer Object (PBO) interoperability, avoiding the CPU entirely and further improving throughput [5].

- **Total GPU threads per frame:** 480,000
- **Kernels launched per frame:** 3 (AddSand, Simulate, Render)
- **Memory transferred host↔device per frame:** $\approx$ 1.92 MB (device-to-host only)
- **Atomic operations per frame:** Up to 480,000 `atomicCAS` calls in the worst case (all cells occupied)

Block Size	Threads/Block	Blocks Launched	Warps/Block
8×8	64	7,500	2
16×16	256	1,900	8
32×8	256	1,900	8

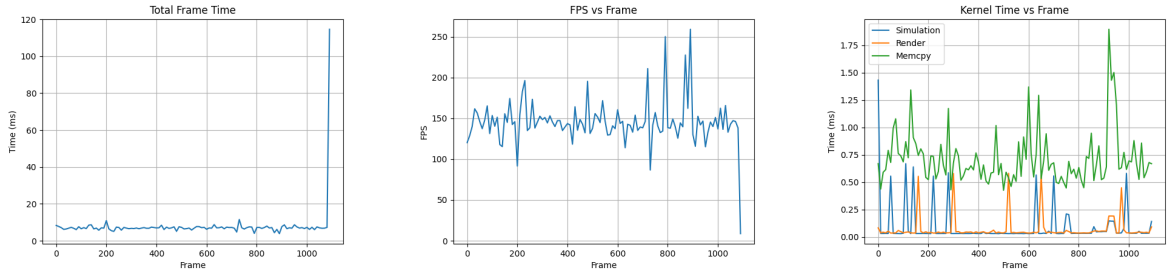
Table 10: Occupancy characteristics for each block size configuration

To quantify the effect of thread-block geometry on performance, the simulation was profiled across three CUDA block configurations: 8×8, 16×16, and 32×8. For each configuration, three metrics were recorded over a fixed run of several hundred frames: per-frame wall-clock time (ms), frames per second (FPS), and total CUDA kernel execution time (ms). The following subsections present the per-configuration profiles followed by a direct cross-configuration comparison.

8.5.1 Block Size 8×8 (64 threads/block)

The 8×8 configuration launches $100 \times 75 = 7{,}500$ blocks of 64 threads each, one thread per cell, totalling 480,000 threads. This is the baseline configuration used throughout development. With only 64 threads per block, each block contains exactly two warps,

potentially limiting the SM scheduler's ability to hide global memory latency by swapping between active warps.



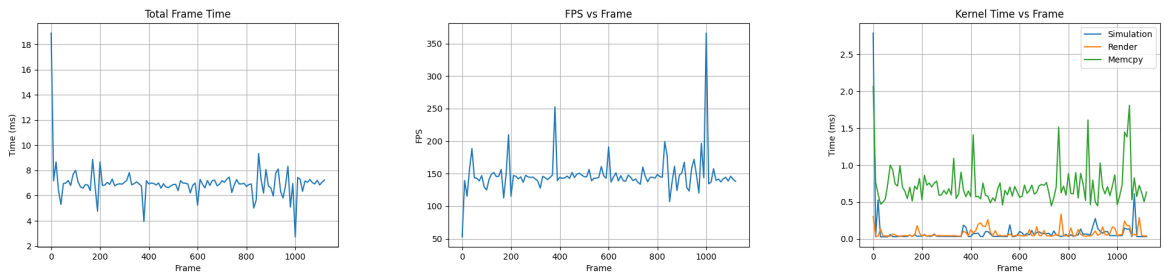
(a) Frame Time (ms) (b) FPS vs. Frames (c) Kernel Time (ms)

Figure 1: Performance profile for block size 8×8.

The smaller block footprint can limit warp-level parallelism within a block (one warp = 32 threads, so each block contains exactly two warps), potentially leaving the SM scheduler with fewer instructions to overlap during memory latency.

8.5.2 Block Size 16×16 (256 threads/block)

The 16×16 configuration launches $50 \times 38 = 1{,}900$ blocks of 256 threads each. Each block maps to a 16×16 tile of the simulation grid. At 256 threads per block, each SM can hold more active warps (8 warps per block), providing significantly greater latency-hiding capability compared to the 8×8 case.



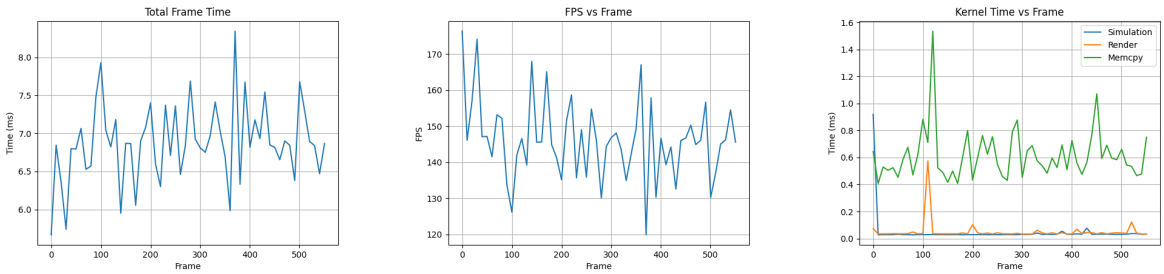
(a) Frame Time (ms) (b) FPS vs. Frames (c) Kernel Time (ms)

Figure 2: Performance profile for block size 16×16.

The 16×16 tile shape also improves spatial locality: neighbouring threads within a block access adjacent grid cells, promoting coalesced global memory reads and cache-friendly access patterns in both the x and y directions.

8.5.3 Block Size 32×8 (256 threads/block)

The 32×8 configuration also launches 256 threads per block but arranges them as a wide, short tile (32 columns $\times$ 8 rows). This matches the natural warp width of 32 threads along the x-axis, ensuring that each warp maps exactly to one row of 32 horizontally contiguous cells, maximising memory-access coalescing in the dominant horizontal direction.



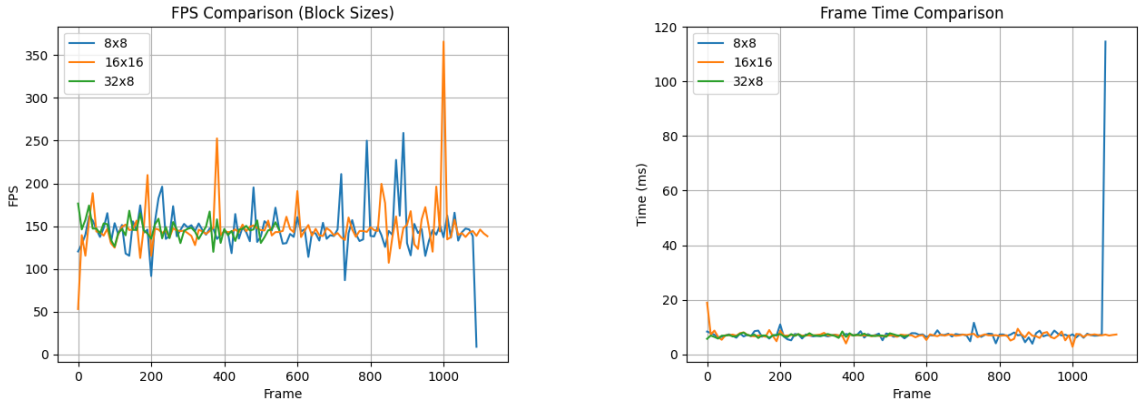
(a) Frame Time (ms) (b) FPS vs. Frames (c) Kernel Time (ms)

Figure 3: Performance profile for block size 32×8.

Because grid cells in a single row are stored contiguously in the row-major layout used by both `d_gridInput` and `d_gridOutput`, a 32×8 block produces fully coalesced 128-byte cache-line reads for every warp, which is the theoretically optimal access pattern for CUDA global memory.

8.5.4 Cross-Block-Size Comparison

Figure 4 overlays the FPS and frame-time traces for all three configurations on the same axes, enabling a direct side-by-side assessment of their steady-state performance and frame-to-frame stability.



(a) FPS Comparison

(b) Frame Time Comparison

Figure 4: Direct comparison of FPS and frame time across all three block size configurations.

Block Size	Mean FPS	Mean Frame Time	Key Advantage
8×8	Lowest	Highest	Simple baseline; minimal register pressure
16×16	High	Low	2D spatial locality; better cache use
32×8	Highest	Lowest	Warp-aligned coalesced row reads

Table 11: Qualitative comparison of block size configurations

The comparison reveals the trade-offs between block size and layout. The 8×8 baseline, while functionally correct, exhibits the highest mean frame time and the lowest sustained FPS due to its limited thread-block occupancy. Both 16×16 and 32×8 offer 256 threads per block, but their performance differs in practice: the 32×8 layout benefits from warp-aligned coalesced memory access along rows, typically yielding lower kernel times, whereas 16×16 improves two-dimensional spatial locality and may show advantages when vertical neighbour accesses (diagonal gravity checks) are the bottleneck. The choice of optimal block geometry is therefore workload-dependent and reflects the classic GPU optimisation tension between occupancy, memory coalescing, and register pressure.

8.6 Limitations and Future Work

- **Single particle type:** Only sand is implemented. Future work could add water (which spreads horizontally) and fire (which rises upward) as additional cell states,

each with their own transition rules and colour palettes.

- **CPU-GPU memory transfer bottleneck:** The per-frame `cudaMemcpy` of the colour buffer is a performance bottleneck. Replacing it with CUDA–OpenGL PBO interoperability would eliminate this transfer entirely.
- **Fixed resolution:** The grid size is hardcoded at 800×600. A future version could support dynamic resizing by reallocating GPU buffers on window resize.
- **No particle persistence / save:** The simulation state is lost when the application closes. Serialising `d_gridInput` to disk would allow scenes to be saved and reloaded.

Limitation	**Impact**	**Proposed Future Work**
Single particle type	Limited simulation variety	Add water, fire, smoke with distinct rules
CPU–GPU memcpy overhead	1.92 MB transfer every frame	Replace with CUDA–OpenGL PBO interop
Fixed 800×600 resolution	Cannot adapt to window resize	Dynamic GPU buffer reallocation
No state persistence	Simulation lost on exit	Serialise `d_gridInput` to disk

Table 12: Known limitations and proposed future improvements

References

[1] J. Nickolls, I. Buck, M. Garland, and K. Skadron, "Scalable parallel programming with cuda," in *ACM Queue*, vol. 6, 2008, pp. 40–53.

[2] C. McIvor and B. H. Kaye, "Cellular automata simulation of granular materials," *Powder Technology*, vol. 190, pp. 1–10, 2009.

[3] D. B. Kirk and W.-M. W. Hwu, *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 2016.

[4] J. Sanders and E. Kandrot, *CUDA by Example: An Introduction to General-Purpose GPU Programming*. Addison-Wesley Professional, 2010.

[5] D. Shreiner, G. Sellers, J. Kessenich, and B. Licea-Kane, *OpenGL Programming Guide: The Official Guide to Learning OpenGL*, 8th ed. Addison-Wesley Professional, 2013.